



Univerzitet u Sarajevu
Elektrotehnički fakultet
Sarajevo

**UNIVERZITET U SARAJEVU**
ELEKTROTEHNIČKI FAKULTET

# P16: BVH RAY TRACER

Izvještaj o naučnom radu

**RAD [2]**

*Parallel Locally-Ordered Clustering for Bounding Volume Hierarchy Construction*

Meister i Bittner (2017/2018)

Predmet: Paralelni računarski sistemi
Mentor: Tarik Hrnjić, MA


**Autori**

Dino Burić - 2502/2026
Ali Fazlić - 2590/2026
Zijad Doglod - 2576/2026
Ajdin Dželo - 2527/19540


Sarajevo, novembar 2026.

## Sažetak

Meister i Bittner predstavljaju Parallel Locally-Ordered Clustering (PLOC), masovno paralelan algoritam koji BVH gradi odozdo prema gore. Klasteri se čuvaju po Morton redoslijedu, pa svaki klaster pronalazi približnog najbližeg susjeda pregledom malog prozora, a svi uzajamno najbliži parovi spajaju se u istoj iteraciji. Na GPU-u PLOC postiže do dvostruko kraće vrijeme izgradnje i do 17% brže praćenje zraka od uporedivih metoda na složenim scenama. Glavni nalaz za P16 je da algoritam ne zavisi od CUDA-e: sastoji se od paralelnih petlji i prefix scan operacija, pa je OpenMP verzija na CPU-u realna opcija.

## 1. Rad ukratko

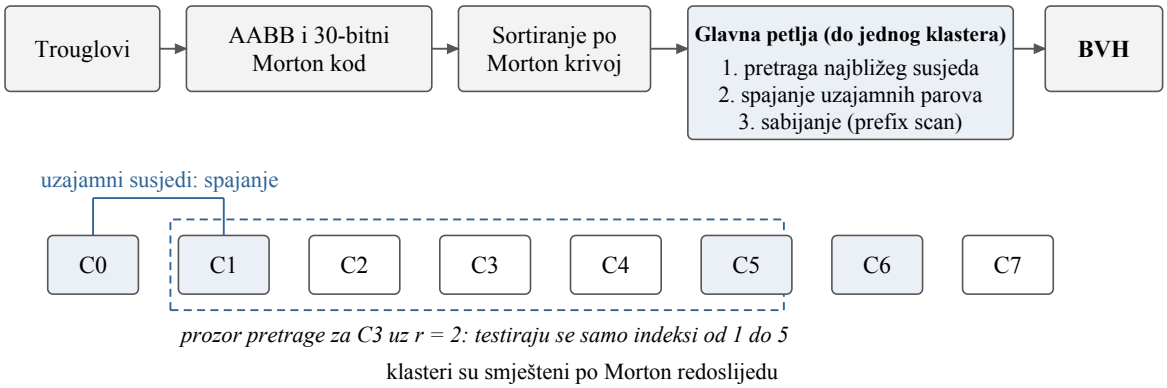
| | |
|---|---|
| **Referenca** | Meister i Bittner, IEEE TVCG, preprint 2017 [2] |
| **Problem** | Kvalitetna izgradnja BVH-a odozdo prema gore je skupa i teško se primjenjivala na višejezgrenom hardveru tipa GPU. |
| **Tehnika** | Lokalno uređeno aglomerativno klasterovanje s približnom pretragom susjeda duž Morton krive. |
| **Hardver** | NVIDIA GTX TITAN X (CUDA 7.5), uz Intel Core i7-3770 i Windows 7. |
| **Glavni rezultat** | Kraća izgradnja i brže praćenje zraka na velikim scenama u odnosu na ATRBVH; do 39% bolje ukupno vrijeme do slike. |
| **Vrijednost za P16** | **Srednja.** Odličan paralelni algoritam za opcionalnu nadogradnju. Nije potreban za V1–V3. |

## 2. Problem koji rad rješava

Izgradnja odozdo prema gore počinje tako što je svaki trougao vlastiti klaster, a zatim se bliski klasteri uzastopno spajaju. Općenito daje BVH visokog kvaliteta, mjereno SAH troškom. Usko grlo je pretraga najbližih susjeda. Naivna pretraga traje $O(n^2)$, a kD-stablo iz ranijih radova pomaže, ali se teško paralelizuje. Prethodni CPU pristupi, poput AAC-a, koriste Morton kodove u pristupu „podijeli pa vladaj“, koji se ne preslikava prirodno na GPU [2].

## 3. Korištena tehnika

PLOC počiva na dvije ideje. Prvo, udaljenost dva klastera definisana je kao površina kutije koja obuhvata oba, a ta funkcija ne može opadati kako klasteri rastu. Ako su dva klastera uzajamno najbliži susjedi, kasnije se ne može pojaviti bolji partner, pa se mogu odmah sigurno spojiti. Zato se svi takvi parovi mogu spajati paralelno. Drugo, klasteri se jednom sortiraju po Morton kodovima svojih centroida. Klaster $i$ zatim testira samo interval od $i-r$ do $i+r$, gdje je $r$ radijus pretrage. Dobiveni susjed je približan, ali uzajamna provjera i dalje štiti odluke o spajanju.




*Slika 1. PLOC tok rada i pretraga susjeda unutar prozora (naš crtež, prema [2]).*

Svaka iteracija ima tri faze odvojene barijerama. U fazi pretrage svaki klaster unutar prozora pronalazi najbližeg susjeda. U fazi spajanja par spaja nit koja obrađuje niži indeks, a drugo mjesto se označava nevažećim. U fazi sabijanja prefix scan uklanja nevažeća mjesta i čuva redoslijed klastera bez ponovnog sortiranja. Petlja se završava kada ostane jedan klaster.

Autori dokazuju završetak algoritma: svaka iteracija spaja barem jedan par, pod uslovom da se kod jednakih udaljenosti bira niži indeks. Teorijska složenost kreće se od $O(rn^2)$ u najgorem do $\Omega(rn)$ u najboljem slučaju, dok je izmjereno ponašanje blizu najboljeg slučaja, linearno i po $r$ i po $n$. Dobiveni BVH ima jedan trougao po listu, pa završni prolaz sažima podstabla u veće listove kada to smanjuje SAH trošak.

## 4. Rezultati

Evaluacija obuhvata devet scena od 331 hiljade do 12,8 miliona trouglova, u poređenju s LBVH, HLBVH i ATRBVH na GPU-u te AAC-om na CPU-u. Tabela 1 prikazuje najmanju i najveću scenu, koje ilustruju najvažniji obrazac. „Ukupno“ je vrijeme izgradnje i renderovanja za path tracing pri rezoluciji 1024×768, s 8 i sa 128 uzoraka po pikselu (spp).

| Scena | Metoda | SAH trošak | Praćenje [MRays/s] | Izgradnja [ms] | Ukupno, 8 spp [ms] | Ukupno, 128 spp [ms] |
|---|---|---|---|---|---|---|
| Conference (331k) | LBVH | 154 | 210 | 3 | 248 | 3932 |
| | ATRBVH | 87 | 287 | 10 | 190 | 2888 |
| | PLOC, r = 25 | 84 | 299 | 22 | 195 | 2784 |
| Power Plant (12,8M) | LBVH | 127 | 55 | 93 | 746 | 10551 |
| | ATRBVH | 81 | 75 | 304 | 775 | 7909 |
| | PLOC, r = 10 | 80 | 84 | 163 | 588 | 6965 |
| | PLOC, r = 25 | 78 | 88 | 210 | 611 | 6699 |
| | PLOC, r = 100 | 77 | 85 | 375 | 793 | 7081 |

*Tabela 1. Izvod iz Tabele 1 rada [2]. Vremena izgradnje su stvarna vremena mjerena na strani CPU-a.*

Na sceni Power Plant PLOC s $r = 25$ gradi oko 31% brže od ATRBVH-a i prati zrake 17% brže, što poboljšava ukupno vrijeme na oba nivoa kvaliteta. LBVH gradi mnogo brže, ali daje stablo slabijeg kvaliteta, pa ukupno gubi čim se prati mnogo zraka. To je jasan primjer kompromisa između vremena izgradnje i vremena praćenja.

Radijus djeluje kao jedan parametar za podešavanje. Vrijeme izgradnje raste linearno s $r$, dok se SAH trošak brzo stabilizuje i za male radijuse. U odnosu na AAC na CPU-u autori navode oko 3 puta kraće vrijeme izgradnje za San Miguel (PLOC s $r = 25$ prema AAC-Fast). AAC je bio sekvencijalan, a njegova vremena su podijeljena s 4 kako bi se dobila optimistična procjena za CPU s 4 jezgre.

## 5. Kritički osvrt

PLOC nije univerzalno bolji. Najbolji je za scene iznad približno milion trouglova. Na manjim scenama dominira trošak upravljanja kernelima: u Tabeli 1 PLOC gradi Conference više nego dvostruko duže od ATRBVH-a. Slabiji je i na scenama tipa jednog objekta, poput Hairballa, gdje je praćenje zraka 16% sporije nego s ATRBVH-om. Uz to, ubrzanja važe za taj GPU i tu verziju CUDA-e, što je naš Checkpoint 1 već naglasio [CP1].

Jedan zaključak iz Checkpointa 1 trebamo proširiti. PLOC smo tretirali kao referencu vezanu za CUDA-u i NVIDIA GPU. Sam algoritam to nije: Algoritam 1 u [2] koristi paralelne petlje, barijere i prefix scan, a autori izričito navode da bi bilo zanimljivo prilagoditi ga višejezgrenim CPU-ima. Međutim, navode i da AAC ostaje dobra opcija za CPU ray tracing, pa ne možemo pretpostaviti da se GPU prednost prenosi. Isplati li se PLOC na CPU-u, moguće je utvrditi samo mjerenjem.

## 6. Šta možemo iskoristiti u P16

Morton kodovi su najupotrebljiviji dio. Daju jeftin prostorni redoslijed koristan za osnovni algoritam izgradnje, a potencijalno i za memorijsku lokalnost. Puna implementacija PLOC-a ostaje nadogradnja, ali predlažemo da se prvo uradi u OpenMP-u, a tek onda razmotri CUDA. Tako bi projekat dobio algoritam izgradnje projektovan za paralelno izvršavanje, kao kontrast rekurzivnoj izgradnji zadacima iz [1], i dobar primjer za diskusiju o barijerama i podatkovnom paralelizmu.

| **Ideja iz [2]** | **Kako je primjenjujemo** | **Prioritet** |
|---|---|---|
| Odvojena izgradnja i praćenje; ukupno vrijeme do slike | Prikazati vrijeme izgradnje, renderovanja i ukupno za barem dva nivoa kvaliteta. | Osnovno |
| Kompromis vremena izgradnje i kvaliteta | Brz i bolji algoritam izgradnje porediti po ukupnom vremenu, a ne samo po izgradnji. | Preporučeno |
| 30-bitni Morton kodovi | Kodove računati u kocki oko scene, a primitive poredati paralelnim sortiranjem. | Preporučeno |
| Radijus pretrage r | Ako se PLOC implementira, ponoviti eksperiment za r = 10, 25 i 100. | Opcionalno |
| PLOC u OpenMP-u | Treća varijanta izgradnje na CPU-u, u poređenju s izgradnjom pomoću zadataka. | Opcionalno |
| Sažimanje podstabala | Listove s jednim trouglom pretvoriti u veće listove prema SAH trošku. | Opcionalno |
| PLOC u CUDA-i | Zaseban GPU korak, samo uz NVIDIA GPU i nakon završetka CPU dijela. | Opcionalno |

*Tabela 2. Plan integracije. Prioritet: **Osnovno** = potrebno za V1–V3; **Preporučeno** = vrijedna nadogradnja; **Opcionalno** = samo ako vrijeme dozvoli.*

```
while (c > 1) {
   #pragma omp parallel
   {
      #pragma omp for schedule(dynamic, 256)        // 1. pretraga; barijera na kraju petlje
      for (int i = 0; i < c; ++i) N[i] = nearestInWindow(in, i, r, c);  // jednakost: najniži indeks

      #pragma omp for schedule(static)              // 2. spajanje; svaki par se obrađuje jednom
      for (int i = 0; i < c; ++i)
         if (N[N[i]] == i && i < N[i]) { in[i] = merge(in[i], in[N[i]]); in[N[i]] = INVALID; }
   }
   c = compactWithPrefixSum(in, out, c);            // 3. sabijanje; čuva Morton redoslijed
   std::swap(in, out);
}
```

*Listing 1. Skica OpenMP verzije glavne PLOC petlje za P16 (naš kod, prema Algoritmu 1 iz [2]).*

Skica je sigurna jer su uzajamni parovi disjunktni, a u njih piše samo nit s nižim indeksom. Funkcija spajanja mora još zauzeti novi indeks čvora, za šta je dovoljan *omp atomic capture* nad brojačem. Pošto je originalna implementacija dostupna na stranici PLOC projekta, detalje možemo s njom uporediti ako nastavimo u tom smjeru.

## 7. Zaključak

PLOC pokazuje da se kvalitetna izgradnja BVH-a može preformulisati u jednostavne podatkovno paralelne faze, a daje i najjasniji dokaz u našoj literaturi da o izboru algoritma izgradnje odlučuje ukupno vrijeme. Za P16 ga ostavljamo van osnovnog obima. Ipak, preporučujemo da se eventualna implementacija prvo usmjeri na OpenMP, a ne na CUDA-u, jer to odgovara predmetu, ne traži poseban hardver i odgovara na pitanje koje rad ostavlja otvorenim.

## Literatura

[1] Waldemarson, G., Doggett, M. (2025). Parallel Axis Split Tasks for Bounding Volume Construction with OpenMP. GRAPP/VISIGRAPP 2025, Vol. 1, str. 347–354. doi:10.5220/0013317100003912.

[2] Meister, D., Bittner, J. (2017/2018). Parallel Locally-Ordered Clustering for Bounding Volume Hierarchy Construction. IEEE TVCG. dcgi.fel.cvut.cz/projects/ploc/ploc-tvcg.pdf.

[CP1] Burić, D., Fazlić, A., Doglod, Z., Dželo, A. (2026). P16: BVH Ray Tracer, Istraživački izvještaj, Checkpoint 1. Univerzitet u Sarajevu, Elektrotehnički fakultet.